



PRÁCTICA 2 : Tareas *en Ada*

OBJETIVOS:

1. Conocer la representación de procesos concurrentes en el lenguaje Ada, implementando tareas para representar distintos elementos de un sistema simulado.
2. Detectar y comprender algunos de los problemas que surgen en la ejecución de un sistema concurrente.

PERIODO RECOMENDADO PARA SU REALIZACIÓN: 1 + ½ semanas

IMPORTANTE! En esta práctica (y las sucesivas con Ada) vamos a usar la librería gráfica GtkAda. En la máquina virtual está instalada la versión 3.24.20. Si estás desarrollando con tu propia instalación de Ada, debes descargarte una versión compatible de esta librería.

ENUNCIADO: Vuelo de aviones

A partir del entorno gráfico proporcionado se debe implementar el vuelo “simultáneo” de aviones. La siguiente figura muestra un ejemplo de la ejecución del sistema simulado completo en un instante determinado.

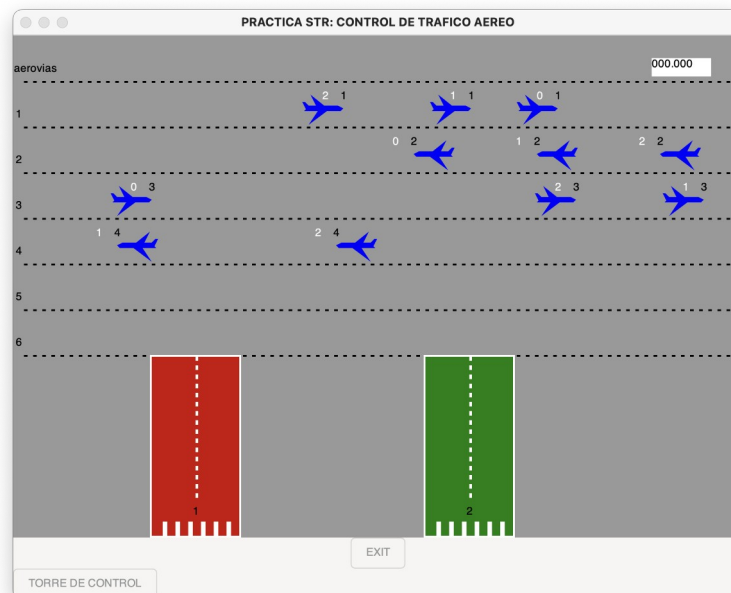


Figura 1. Ejemplo de ejecución del sistema simulado. En la parte superior de cada avión, se muestra su identificador en la aerovía (en color blanco) y el número de aerovía por la que ha accedido al espacio aéreo (en color negro).



Tenemos 6 aerovías por las que vuelan aviones a distintas altitudes. La dirección de vuelo es hacia la derecha en las aerovías impares y hacia la izquierda en las pares. Los aviones aparecen por el margen correspondiente de la aerovía y cuando llegan al final vuelven al principio de la misma (simulamos un vuelo circular en cada aerovía). De momento los aviones no pueden cambiar de aerovías, con el fin de ir descendiendo hasta aterrizar en las pistas. Esto lo implementaremos en futuras prácticas.

En la parte superior derecha aparece un cronómetro que no funciona. Lo implementaremos en la siguiente práctica. El botón denominado “TORRE DE CONTROL”, tampoco tendrá ninguna funcionalidad de momento, y si se pulsa simplemente desaparece del entorno.

En esta práctica debes implementar la aparición de 3 aviones en cada una de las 4 primeras aerovías, que irán volando por su aerovía hasta que el usuario finalice la ejecución del programa al pulsar el botón EXIT.

En la imagen mostrada en la Figura 1, hay 3 aviones en cada aerovía excepto en la última, en la que falta uno, debido a que al intentar acceder a la misma, el espacio al que debía acceder estaba ocupado por otro avión. En el paso 3 se explica cómo gestionar esta situación.

Cada avión que aparece tiene que ser representado por una tarea distinta.

PASOS A SEGUIR:

En primer lugar, se detalla una breve descripción de los paquetes proporcionados que deberás incluir en tu proyecto:

- **Paquetes** que se deben usar como **ayuda** para implementar el proyecto y que en los enunciados de las prácticas, se irá indicando en cada caso qué usaremos de cada uno:
 - **pkg_tipos** : declaración de constantes y tipos de datos (**sólo el fichero .ads**)
 - **pkg_graficos** : procedimientos y funciones para la representación gráfica
 - **pkg_debug** : **paquete** para imprimir en un fichero de texto y por pantalla mensajes para facilitar la **depuración** del programa si se considera necesario:
 - sólo necesitarás usar el procedimiento **Escribir**
 - Averigua cómo se llama el fichero de texto donde se escriben los mensajes de salida.
 - ¿Por qué no tienes que invocar al procedimiento **Crear_Fichero**?
 - **pkg_torre_control** : **paquete** usado desde **pkg_graficos** para gestionar el botón correspondiente y que modificaremos en otra práctica.
 - **pkg_buffer_generico** : **paquete** usado desde **pkg_graficos** y que sólo lo necesitarás para la **compilación** del proyecto.

IMPORTANTE! NO debes modificar los paquetes proporcionados por el profesor. **Debes crear tus propios ficheros** en los que se definan paquetes que incluyan tu código fuente que implementes.



PASO 1: Visualizar el entorno estático simulado

Crea un proyecto denominado **practica2** utilizando la plantilla “*Basic → Simple Ada Project*” y copia en su carpeta /src todos los paquetes proporcionados.

El cuerpo del programa principal que debes implementar debe contener lo siguiente:

```
begin
  pkg_graficos.Simular_Sistema;
end main;
```

Aclaración: el procedimiento *pkg_graficos.Simular_Sistema* se encarga de la animación gráfica. Implementa un bucle infinito que redibuja y captura eventos de la interfaz gráfica. Se interrumpe al pulsar el botón **Exit** de la ventana principal, acabando automáticamente la ejecución del programa principal y de todas las tareas en ejecución.

Antes de compilar el proyecto, hay que añadir como dependencia del mismo la librería gráfica *gtkAda*. Para ello, desde el menú contextual del proyecto utiliza la opción *Project → Properties → Dependencies* y pulsando el botón '+' selecciona el fichero *gtkada.gpr* que se encuentra en la siguiente ruta de la máquina virtual: */usr/gnat/lib/gnat/*. El campo “Limited with” no hay que activarlo

Al finalizar el paso 1, el resultado de la ejecución de tu proyecto deberá ser simplemente la visualización de la ventana principal del entorno simulado. Tu programa finalizará su ejecución cuando pulses el botón Exit.

PASO 2: Creación de aviones

Utiliza el body de la tarea *TareaGeneraAviones* y del tipo tarea *T_Tarea_Avion* del ejercicio 8 de la práctica 1, para crear un nuevo paquete con las funcionalidades necesarias para la creación de aviones en el sistema simulado. En este paso, imprimiremos en el log los datos de cada avión creado, y en el siguiente paso los visualizaremos en el entorno gráfico.

Implementa los siguientes requerimientos:

1. Debe aparecer un nuevo avión en una aerovía con una frecuencia de aparición aleatoria. Para ello genera un valor aleatorio de retardo de espera y usa la sentencia: *Delay(Duration(retardo))* ; Esta debe ser la última sentencia del bucle, donde *retardo* representa el valor aleatorio generado de tipo *T_RetardoAparicionAviones* y se realiza un casting al tipo *Duration* (expresado en segundos) para que la tarea suspenda su ejecución el tiempo especificado.



2. Se deben crear un número constante de aviones en las 4 primeras aerovías (ya lo tienes implementado en el código proporcionado de la práctica 1).
3. Debes inicializar la información restante de cada avión, ya que el nuevo tipo record proporcionado contiene más campos que el mostrado en la práctica 1, y deberás actualizar también dichos campos. En concreto, la aerovía inicial, la pista asignada (con valor cero para denotar que no tiene pista), un color, la dirección de vuelo (velocidad en eje X positiva si el número de aerovía es impar y negativa si es par), y por último la posición de inicio en la que aparecerá el avión.
4. En el cuerpo del tipo tarea `T_Tarea_Avion`, añade la información restante del avión para que se imprima en el log.
5. Incorpora en el cuerpo de las tareas el siguiente manejador de excepciones que imprimirá en el log de nuestro programa una posible terminación anormal de la tarea a causa de una excepción no manejada:

```
exception  
when event: others =>
```

```
    PKG_debug.Escribir("ERROR en TASK ....: " & Exception_Name(Exception_Identity(event)));
```

Del paquete `pkg_tipos`, debes utilizar lo siguiente:

- el tipo `T_RecordAvion`, para la información de un avión
- el tipo `Ptr_T_RecordAvion`, para pasar como parámetro discriminante la información del avión a la tarea
- el subtipo `T_RetardoAparicionAviones`, que define el rango de segundos para obtener valores aleatorios de la frecuencia de aparición
- del subtipo `T_ColorAvion`, usaremos el color Blue para asignarlo a todos los aviones
- el tipo `T_IdAvion`, que define el rango de número de identificación de un avión en una aerovía
- el tipo `T_Rango_Aerovia`, que define el rango números de aerovías
- constante `VELOCIDAD_VUELO` (velocidad constante en el eje X de un avión)

Del paquete `pkg_graficos`, utiliza lo siguiente:

- *Function* `Pos_Inicio`, que es una función sobrecargada y tendrás que indicar como parámetro la aerovía correspondiente del avión. La función devuelve la posición (x,y) en la que debe aparecer el avión en la aerovía especificada.

Al finalizar el paso 2, se debe haber añadido la siguiente funcionalidad a tu proyecto: la creación con una frecuencia aleatoria de un nuevo avión en una aerovía. Cada avión solo mostrará en el log su información, pero de momento no se hará visible en el sistema

IMPORTANTE! Para utilizar los procedimientos y funciones del paquete `pkg_graficos`, en los enunciados de las prácticas se indicará cuáles debes utilizar en cada momento y tendrás una descripción de lo que hacen. Por tanto, no necesitas consultar cómo están implementados (fichero con extensión `.adb`), sino sólo tendrás que consultar su especificación (fichero con extensión `.ads`) para comprobar los tipos de datos de los parámetros y el tipo devuelto por la función en su caso.



PASO 3: Vuelo de aviones

Modifica el body del tipo tarea T_Tarea_Avion con las funcionalidades necesarias para el vuelo de un avión en una aerovía, que vienen dadas por los siguientes requerimientos:

1. El avión aparecerá en su aerovía asignada.
2. El avión se moverá con una velocidad constante por la aerovía.

Para simular el movimiento de los aviones, simplemente se debe invocar al procedimiento **pkg_graficos.Actualiza_Movimiento** en un bucle infinito, junto con la sentencia **delay(pkg_tipos.RETARDO_MOVIMIENTO)** que hará que se invoque a dicho procedimiento cada 0.1 segundos. Dependiendo de la velocidad del avión, el sistema calculará automáticamente su nueva posición.

Del paquete **pkg_tipos**, debes utilizar lo siguiente:

- constante **RETARDO_MOVIMIENTO**, explicado su uso en el párrafo anterior

Del paquete **pkg_graficos**, utiliza lo siguiente:

- Procedure **Aparece**, que hace visible el avión indicado como parámetro
- Procedure **Actualiza_Movimiento**, que actualiza la posición del avión según su velocidad
- Procedure **Desaparece**, que deja de visualizar el avión indicado como parámetro

Ten en cuenta que cuando aparece un nuevo avión al principio de su aerovía, puede haber otro avión cercano a esa posición. En ese caso, el sistema automáticamente generará la excepción **pkg_tipos.DETECTADA_POSIBLE_COLISION** al invocar al procedimiento **Actualiza_Movimiento**. Debes tratar esta excepción de forma que se muestre en el log el identificador del avión que no puede acceder a la aerovía y hacerlo desaparecer del sistema (invocando al procedure **Desaparece**) y terminando la ejecución de esa tarea.

Al finalizar el paso 3, se deberá haber añadido la siguiente funcionalidad a tu proyecto: la visualización del vuelo de 3 aviones (**pkg_tipos.NUM_INICIAL_AVIONES_AEROVIA**) en las 4 primeras aerovías, a menos que se detecte alguna posible colisión, en cuyo caso aparecerá en el log el identificador del avión que no ha podido acceder a su aerovía. Los aviones volarán - ininterrumpidamente por su aerovía, hasta que el usuario pulse la tecla EXIT.